PROGRAMMING IN JAVA

Assignment 03

TYPE OF QUESTION: MCQ

Number of questions: 10 Total marks**: 10 × 1 = 10**

QUESTION 1:

Which of the following statements is true regarding the order of execution of constructors in an inheritance hierarchy?

a. **Derived class constructor will be called followed by the base class constructor.**

b. **Only Base class constructor will be called.**

c. **Only derived class constructor will be called.**

d. **Base class constructor will be called followed by the derived class constructor.**

Correct Answer: d

d. **Base class constructor will be called followed by the derived class constructor.**

Detailed Solution:

On object creation of a derived class, Java ensures that the base (parent) class constructor is executed first to initialize the parent's state before executing the derived class constructor.

QUESTION 2:

The super() method is used to:

a. **Call constructor of friend class**

b. **Call constructor of the parent class**

c. **Call constructor**

d. **Is a declared method**

Correct Answer: b

b. **Call constructor of the parent class**

Detailed Solution:

In Java, `super()` is used to invoke the constructor of the immediate parent class. It allows the subclass to initialize the parent class's state.

QUESTION 3:

What is the output of the following Java program?

```
class Vehicle {
    public void move() {
        System.out.println("The vehicle moves");
    }
}

class Car extends Vehicle {
    public void move() {
        System.out.println("The car moves");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle vehicle = new Car();
        vehicle.move();
    }
}
```

a. **"The vehicle moves"**

b. **"The car moves"**

c. **The code does not compile**

d. **None of the above**

Correct Answer: b

b. **"The car moves"**

Detailed Solution:

This is an example of method overriding. Since the object is instantiated as a `Car`, the overridden `move` method in the `Car` class is executed, even though the reference type is `Vehicle`.

QUESTION 4:

Which of the following is NOT true regarding the final keyword in Java?

a. **A final method cannot be overridden in a subclass.**

b. **A final variable can only be assigned once.**

c. **A final class can have subclasses.**

d. **A final variable can be assigned during declaration or in the constructor.**

Correct Answer: c

c. **A final class can have subclasses.**

Detailed Solution:

A class marked as `final` cannot be extended, meaning it cannot be subclassed (it cannot have any child classes).

QUESTION 5:

Which type of inheritance leads to the "Diamond Problem"?

a. **Single level**

b. **Multi-level**

c. **Multiple**

d. **Hierarchical**

Correct Answer: c

c. **Multiple**

Detailed Solution:

The diamond problem arises in multiple inheritance when a class inherits from two classes that have a common ancestor, leading to ambiguity regarding which path to follow for inherited members. Java avoids this by not supporting multiple inheritance with classes.

QUESTION 6:

Members of a class which are not intended to be inherited by subclasses should be declared as:

a. **Public members**

b. **Protected members**

c. **Private members**

d. **Private or Protected members**

Correct Answer: c

c. **Private members**

Detailed Solution:

The `private` access specifier is the most restrictive. Private members are not visible to subclasses and therefore are not inherited.

QUESTION 7:

In Java, is it possible to override a static method?

a. **Yes, we can override a static method just like we do with instance methods.**

b. **No, static methods cannot be overridden because they belong to the class, not the object.**

c. **It depends on whether the static method is declared as final or not.**

d. **It depends on the access modifier of the static method.**

Correct Answer: b

b. **No, static methods cannot be overridden because they belong to the class, not the object.**

Detailed Solution:

Static methods are bound to the class rather than the instance (static binding). If a subclass defines a static method with the same signature, it "hides" the parent method rather than overriding it.

QUESTION 8:

What is the output of the following Java program?

```
public class NptelExample {
    public static int fun(int n) {
        if (n == 0) {
            return 1;
        }
        return n * fun(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(fun(5));
    }
}
```

a. 5

b. 24

c. 120

d. Runtime Error

Correct Answer: c

c. 120

Detailed Solution:

The `fun` method is a recursive function calculating the factorial of `n`. For input 5, it calculates 5 * 4 * 3 * 2 * 1 = 120.

QUESTION 9:

Which of the following statements about abstract classes in Java is correct?

a. **Abstract classes can be instantiated directly.**

b. **An abstract class must contain at least one abstract method.**

c. **A class inheriting from an abstract class must implement all its abstract methods unless it is itself abstract.**

d. **Abstract classes can be marked as final.**

Correct Answer: c

c. **A class inheriting from an abstract class must implement all its abstract methods unless it is itself abstract.**

Detailed Solution:

Abstract classes cannot be instantiated. Any concrete subclass (non-abstract) must provide implementations for all abstract methods defined in the parent abstract class.

QUESTION 10:

What will be the output of the below program?

```
class superDemoClass {
    final int a = 20;
}

class subDemoClass extends superDemoClass {
    void subFunc() {
        a = 40;
        System.out.println("value of a = " + a);
    }
}

class demo {
    public static void main(String[] args) {
        subDemoClass subc = new subDemoClass();
        subc.subFunc();
    }
}
```

a. **value of a = 20**

b. **error: cannot assign a value to final variable a**

c. **error: unknown variable 'a' in class subDemoClass**

d. **value of a = 40**

Correct Answer: b

b. **error: cannot assign a value to final variable a**

Detailed Solution:

The variable `a` is declared as `final` in the superclass. While it is inherited, its value cannot be modified (re-assigned) by the subclass, causing a compilation error.